

Exercise 3 - Monte Carlo Simulation of areas

Florescu Andrei - 12242987

2024

Setup

```
set.seed(12242987)
```

Task 1

1. Approximate the integral for $b = 6$ (using Monte Carlo integration).

```
a <- 1
b <- 6
n <- 100000
random_vars <- runif(n, min = a, max = b)

function_to_integrate <- function(x) {
  return(exp(-x^2))
}
```

Monte Carlo aproximation

```
monte_carlo_aprox <- (b - a) * mean(function_to_integrate(random_vars))
print(monte_carlo_aprox)
```

```
## [1] 0.138051
```

Result using integrate function

```
integrate_result <- integrate(function_to_integrate, lower = a, upper = b)$value
print(integrate_result)
```

```
## [1] 0.1394028
```

2. Compute the integral for $b = \infty$

We can rewrite the integral

$$\int_1^{\infty} e^{-x^2} dx = \int_0^{\infty} e^{-(x+1)^2} dx$$

then use the exponential distribution to aproximate the integral

```
X <- rexp(n)
monte_carlo_aprox <- mean(exp(-(X + 1)^2) / dexp(X))
print(monte_carlo_aprox)
```

```
## [1] 0.1391722
```

Aproximation using integrate function

```
integrate(function_to_integrate, 1, Inf)
```

```
## 0.1394028 with absolute error < 8.4e-05
```

3. Do you have an explanation why Monte Carlo integration agrees in 2. with integrate but not so much in 1.?

The integral in the first example is close to the integrate function to the second decimal and the integral in the second example is close to the third decimal, which is not a huge difference but the reason might be due to the Strong Law of Large Numbers

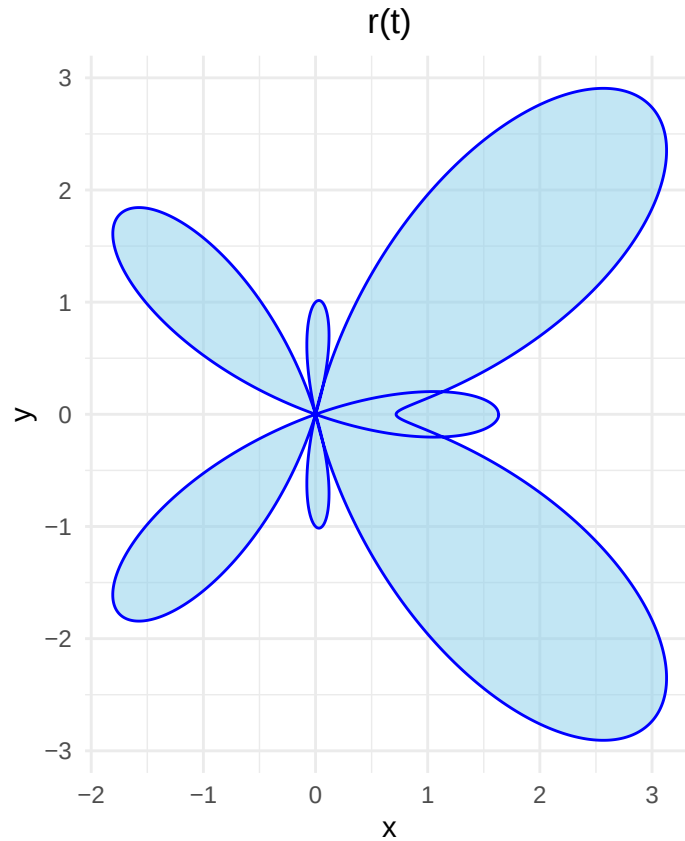
Task 2

1. $r(t)$ function plot

```
library(ggplot2)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.2
```

```
r_func <- function(t) {  
  return(exp(cos(t)) - 2 * cos(4 * t) - (sin(t / 12))^5)  
}  
t_values <- seq(-pi, pi, length.out = 10000)  
  
polar_coord <- data.frame(  
  t = t_values,  
  x = r_func(t_values) * cos(t_values),  
  y = r_func(t_values) * sin(t_values)  
)  
  
ggplot(polar_coord, aes(x = x, y = y)) +  
  geom_polygon(fill = "skyblue", color = "blue", alpha = 0.5) +  
  coord_fixed() +  
  theme_minimal() +  
  labs(  
    title = "r(t)",  
    x = "x",  
    y = "y"  
  ) +  
  theme(plot.title = element_text(hjust = 0.5))
```



2. Generate random coordinates within the rectangle $[-2, 3.5] \times [-3, 3]$

```
n <- 1000 # Number of points
x <- runif(n, min = -2, max = 3.5)
y <- runif(n, min = -3, max = 3)

library(dplyr)

## Warning: package 'dplyr' was built under R version 4.4.2
##
## Attaching package: 'dplyr'
## The following objects are masked from 'package:stats':
##
##   filter, lag
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

points_data <- data.frame(
  x = x,
  y = y
)

points_data <- points_data %>% mutate(
```

```

t = atan2(y, x), # calculate t
r_point = sqrt(x^2 + y^2), # calculate radius for each point
r_boundary = r_func(t), # calculate boundary radius at angle t
is_in_area = r_point <= r_boundary # check if point is in func area
)
head(points_data)

```

```

##           x           y           t  r_point r_boundary is_in_area
## 1  0.8763529 -1.2293905 -0.9515079 1.509767  3.361382      TRUE
## 2  1.4288213 -2.2501587 -1.0050476 2.665473  2.985705      TRUE
## 3 -0.9013321  2.0228081  1.9899723 2.214532  0.876936     FALSE
## 4 -0.1546625  2.8365306  1.6252676 2.840744 -1.005747     FALSE
## 5 -0.9925382  0.7784156  2.4765185 1.261373  2.227697      TRUE
## 6  0.3363601 -0.1268291 -0.3605784 0.359477  2.292720      TRUE

```

3. Simulate 100, 1000, 10000 and 100000 random coordiantes and calculate the percentage of the points within the enclosed area.

```

get_points_percentages <- function(n) {
  x <- runif(n, min = -2, max = 3.5) # get n points
  y <- runif(n, min = -3, max = 3)
  points_data <- data.frame(
    x = x,
    y = y
  )
  # determine if points are in function area
  points_data <- points_data %>% mutate(
    t = atan2(y, x), # calculate t
    r_point = sqrt(x^2 + y^2), # calculate radius for each point
    r_boundary = r_func(t), # calculate boundary radius at angle t
    is_in_area = r_point <= r_boundary # check if point is in func area
  )
  percentage_inside <- table(points_data$is_in_area)
  return(
    percentage_inside["TRUE"] /
      (percentage_inside["TRUE"] + percentage_inside["FALSE"])
  )
}

```

```
get_points_percentages(n = 100)
```

```
## TRUE
## 0.36
```

```
get_points_percentages(n = 1000)
```

```
## TRUE
## 0.357
```

```
get_points_percentages(n = 10000)
```

```
## TRUE
## 0.3861
```

```
get_points_percentages(n = 100000)
```

```
## TRUE  
## 0.38408
```

We can assume that the area is about 0.38 of the rectangle area defined by the limits $[-2, 3.5] \times [-3, 3]$ which is:

```
(3.5 + 2) * (3 + 3) * 0.38
```

```
## [1] 12.54
```

```
t <- seq(-pi, pi, length.out = 1000)  
curve_data <- data.frame(  
  x = r_func(t) * cos(t),  
  y = r_func(t) * sin(t)  
)  
results <- lapply(c(100, 1000, 10000, 100000), get_points_percentages)  
results_df <- do.call(rbind, lapply(results, as.data.frame))  
rownames(results_df) <- c("100", "1000", "10000", "100000")  
results_df["Area"] <- results_df["X[[i]]"] * (3.5 + 2) * (3 + 3)  
colnames(results_df) <- c("Percentage", "Area")  
results_df
```

```
##      Percentage      Area  
## 100         0.2800  9.2400  
## 1000        0.3790 12.5070  
## 10000       0.3895 12.8535  
## 100000      0.3849 12.7017
```

4. Explain the functionality of Monte Carlo simulation in your own words referring to these simulations

The Monte Carlo simulation is a great tool that can be used to approximate calculations in scenarios where direct computation might be difficult to perform (for example the integral mentioned in task 1). It is also very useful in approximating areas of complex functions by defining a simpler region that contains the entire area of the function and then estimate the area of the desired function by checking the percentage of points inside the shape. The simulation result converges towards the real value when the number of generations is large enough and therefore it is much reliable.